

Documentos firmados — API de Sovra

Documentación de integración, endpoint por endpoint. Para cada uno: qué hace, cómo se llama, qué devuelve —campo por campo— y con qué errores puede fallar.

Índice

Contexto

1. [Qué es un documento firmado](#)
2. [Cómo funciona la API](#)
3. [El flujo de punta a punta](#)

Endpoints

4. [POST /api/v1/issuer/documents](#) — Emitir un documento
5. [PUT /api/v1/issuer/documents/{id}/visibility/{visibility}](#) — Publicar o despublicar
6. [GET /api/v1/issuer/documents](#) — Listar los documentos del workspace
7. [GET /api/v1/documents/{id}](#) — Leer un documento (público)

Transversal

8. [El campo `credential` en detalle](#)
 9. [Visibilidad: qué restringe y qué no](#)
 10. [Verificación: por qué no hay un endpoint](#)
 11. [Tabla completa de errores](#)
 12. [Colección Postman](#)
-

Contexto

1. Qué es un documento firmado

Una credencial **sin holder**: el emisor la firma y la entrega. No hay wallet, no hay OID4VCI, no hay flujo de aceptación por parte del destinatario.

La firma queda **anclada on-chain**. Eso cambia dónde vive la confianza: quien recibe el documento lo verifica contra la cadena, no preguntándole al servicio que lo emitió. El servicio puede caerse, cambiar de dominio o dejar de existir, y el documento sigue siendo verificable.

En una frase: **el documento es un string autocontenido; la API es solo la forma de producirlo y de repartirlo**. Ese string es el campo `credential` que aparece en casi todas las respuestas de esta documentación.

2. Cómo funciona la API

Base URL

```
https://test-api-sovra.flagonsa.com
```

En los ejemplos aparece como \$API.

```
export API=https://test-api-sovra.flagonsa.com
export API_KEY=sovra_sk_...
```

Las dos zonas de la API

La API tiene dos zonas con reglas de autenticación distintas, y la diferencia está en el path:

Zona	Path	Autenticación
Emisor	/api/v1/issuer/...	Authorization: Bearer <API_KEY>
Pública	/api/v1/documents/...	Ninguna. Para documentos públicos, tener el id es toda la autorización

Dos detalles que producen 401 en la primera integración:

- **Bearer se compara de forma literal.** bearer en minúsculas devuelve 401 invalid_api_key.
- **Una API key no sustituye una sesión.** Un documento privado leído por la zona pública devuelve 401 authentication_required incluso si se manda la key en el header. Los documentos privados solo se abren con sesión del dashboard.

La API key (sovra_sk_...) es un secreto de servidor. No debe viajar en JavaScript servido al navegador, ni en un repositorio, ni en una colección Postman exportada.

Los cuatro endpoints

#	Método	Path	Qué hace	Auth
4	POST	/api/v1/issuer/documents	Emite: valida, firma y ancla	API key
5	PUT	/api/v1/issuer/documents/{id}/visibility/{visibility}	Publica o despublica	API key
6	GET	/api/v1/issuer/documents	Lista todo el workspace	API key
7	GET	/api/v1/documents/{id}	Devuelve el documento en JSON	—

No hay más. En particular: **no hay endpoints para crear schemas o layouts** (eso es dashboard), **ni para buscar, ni para verificar** (ver §10).

Prerrequisitos del workspace

- Un **schema de tipo document** ya definido en el dashboard. El schema es el molde: declara qué claims existen, de qué tipo son y cuáles son obligatorios. Un schema de **kind: "credential"** (credenciales con holder) no sirve acá → [422 wrong_schema_kind](#).
- Un **layout** para ese schema (opcional). Define cómo se dibuja la hoja. Sin layout la emisión funciona igual; lo que se pierde es el renderizado.
- El workspace debe estar **provisionado**, con DID y claves. Si no lo está → [409 workspace_not_provisioned](#).

3. El flujo de punta a punta

Dashboard	API	Destinatario
a. Definir schema b. Diseñar layout (opcional)	c. POST /issuer/documents → valida, firma, ancla → devuelve el credential d. PUT .../visibility/public (opcional)	\$4 \$5 e. Recibe el credential o el enlace f. Verifica contra la cadena

- **a y b** se hacen desde el dashboard y requieren sesión. No hay API para esto.
- **c** es una sola llamada que valida, firma y ancla.
- **d** es opcional y deliberada: todo documento nace privado.
- **e** puede ser el string `credential` o el enlace `/api/v1/documents/{id}`. Los dos caminos terminan en lo mismo.
- **f** corre del lado de quien recibe, contra la cadena.

Endpoints

4. POST /api/v1/issuer/documents — Emitir un documento

Qué hace

Emite un documento a partir de un schema existente. **Una sola llamada valida los claims, firma con la clave del emisor y ancla la firma on-chain.**

Por eso la latencia no es la de un POST normal: la documentación advierte hasta **~90 segundos**. Conviene poner el timeout del cliente por encima de eso (120 s) y mostrar progreso al usuario. En el ambiente de test la respuesta suele llegar en ~3 segundos, pero el techo depende de la congestión de la cadena, no del caso feliz.

Autenticación

Authorization: Bearer <API_KEY>

Request

Content-Type: application/json

Campo	Tipo	Obligatorio	Qué es
<code>schema_id</code>	<code>string</code>	Sí	El identificador legible del schema (el <code>schema.schema_id</code>), no el UUID interno
<code>claims</code>	<code>object</code>	Sí	Un par <code>key: valor</code> por cada claim del schema. Los <code>required</code> no pueden faltar
<code>visibility</code>	<code>string</code>	No	"private" (default) o "public"

Los tipos de cada claim los declara el schema (`date`, `number`, `string`, `boolean`) y se validan durante la emisión.

```
curl -s -X POST $API/api/v1/issuer/documents \  
-H "Authorization: Bearer $API_KEY" \  
-H 'Content-Type: application/json' \  
-d '{  
  "schema_id": "<schema-id>",  
  "claims": { "<claim_key>": "<valor>" },  
  "visibility": "private"  
}
```

Response — 201 Created

```
{  
  "id": "<uuid>",  
  "status": "anchored",  
  "visibility": "private",  
  "credential": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3Nja2kiLCJ0eXAiOiJKV1QiLCJkaXIiOiJmcm9udC91b250IiwiaWF0IjoxNjU0MjU0MjU0LCJpc3M...~",  
  "claims": { "<claim_key>": "<valor>" },  
  "digest": "<sha256 en hex del signing input>",  
  "tx_hash": "0x...",  
  "signed_at": "...",  
  "anchored_at": "...",  
  "revoked_at": null,  
  "issuer": { "did": "did:sovra:0x...", "name": "...", "address": "0x..." },  
}
```

```
"schema": { "id": "...", "schema_id": "...", "credential_type": "..." }
}
```

Qué responde, campo por campo

La columna **Firmado** indica si el dato está cubierto por la firma del emisor (vive dentro del **credential**) o si es una afirmación del servicio. Ver [§8](#).

Campo	Tipo	Qué es	Firmado
id	string (uuid)	El identificador del documento en el servicio. Se usa en las URLs (<code>/api/v1/documents/<id></code>) y coincide con el claim jti de adentro del credential, así que una credential no puede servirse bajo otro id	Sí (como jti)
status	string	El ciclo de vida. anchored = firmado y con la firma registrada on-chain: es el estado final exitoso	No
visibility	string	public o private . Controla el acceso vía la API — ver §9	No
credential	string	★ El documento en sí . Un SD-JWT VC en base64url, autocontenido y verificable por cualquiera que lo tenga. Es el único campo que hay que conservar	Es la firma
claims	object	Los mismos claims ya decodificados, por conveniencia de la integración. No es prueba de nada: es lo que el servicio dice que dice el documento	Sí, adentro del credential
digest	string (hex)	El SHA-256 del <i>signing input</i> (los bytes que se firmaron). La huella del documento: la verificación lo recalcula y compara. Si alguien altera un byte, deja de coincidir	No (se recalcula)
tx_hash	string (0x...)	El hash de la transacción on-chain donde quedó anclada la firma	No
signed_at	string (ISO 8601)	Cuándo el emisor firmó	Sí (como iat)
anchored_at	string (ISO 8601)	Cuándo la firma quedó registrada en la cadena. En las pruebas, segundos después de firmar	No
revoked_at	string null	null = vigente. Con fecha, el documento está revocado	No
issuer	object	Quién emitió — ver abajo	Parcial
schema	object	El molde usado — ver abajo	Parcial

issuer

Campo	Tipo	Qué es	Firmado
<code>issuer.did</code>	<code>string</code>	El identificador descentralizado (<code>did:sovra:0x...</code>). Con este DID el verificador resuelve la clave pública del emisor desde el registry on-chain	Sí (como <code>iss</code>)
<code>issuer.address</code>	<code>string</code> (0x...)	La dirección Ethereum-style del emisor en la cadena, derivada de su clave pública	Sí, se deriva de <code>iss</code>
<code>issuer.name</code>	<code>string</code>	Nombre visible del emisor. Afirmación del servicio — un verificador no debería usarlo para decidir	No

schema

Campo	Tipo	Qué es	Firmado
<code>schema.id</code>	<code>string</code> (uuid)	UUID interno del schema en el servicio	No
<code>schema.schema_id</code>	<code>string</code>	El identificador legible, el que se manda al emitir	No
<code>schema.name</code>	<code>string</code>	Nombre humano del molde	No
<code>schema.version</code>	<code>string</code>	Versión del molde	No
<code>schema.credential_type</code>	<code>string</code>	El tipo de credencial. Viaja como <code>vct</code> dentro del JWT firmado, y es contra este valor que se comprueba on-chain si el emisor está autorizado a emitirlo	Sí (como <code>vct</code>)
<code>schema.claims[]</code>	<code>array<object></code>	La definición de cada campo — ver abajo	No

`schema.claims[]` — cada entrada:

Campo	Tipo	Qué es
<code>key</code>	<code>string</code>	Nombre del campo dentro del objeto <code>claims</code>
<code>label</code>	<code>string</code>	Etiqueta legible, para UI y layout
<code>type</code>	<code>string</code>	Tipo de dato: <code>date</code> , <code>number</code> , <code>string</code> , <code>boolean</code>
<code>required</code>	<code>boolean</code>	Si es <code>true</code> y falta al emitir → <code>422 missing_claims</code>
<code>disclosable</code>	<code>boolean</code>	Si el campo es divulgable selectivamente bajo SD-JWT

Conservar el `credential`

De todo lo que devuelve este endpoint, `credential` es lo único que importa guardar. Todo el resto es la afirmación del propio servicio sobre él y se puede volver a pedir —o perder— sin consecuencias. Si la integración persiste un solo campo, es ese.

Errores de este endpoint

Respuesta	Causa
401 invalid_api_key	Key desconocida, malformada o revocada, o el esquema no era literalmente Bearer
409 workspace_not_provisioned	El workspace todavía no tiene DID ni claves
422 wrong_schema_kind	El schema es kind: "credential" (con holder), no "document"
422 missing_claims	Falta un claim requerido. La respuesta lista cuáles
422 invalid_visibility	visibility no es public ni private
503 chain_not_configured	No hay registro de firmas configurado: se niega a firmar . Es intencional — prefiere fallar antes que emitir algo no anclable

5. PUT

/api/v1/issuer/documents/{id}/visibility/{visibility} — Publicar o despublicar

Qué hace

Cambia la visibilidad del documento. Un documento nace **privado**, así que publicarlo es siempre una decisión deliberada.

Autenticación

Authorization: Bearer <API_KEY>

Request

Sin body y sin Content-Type: los dos valores viajan en el path.

Parámetro	Posición	Tipo	Qué es
id	path	string (uuid)	El id que devolvió la emisión
visibility	path	string	public o private. Cualquier otro valor → 422 invalid_visibility

```
# publicarlo
curl -s -X PUT -H "Authorization: Bearer $API_KEY" \
  $API/api/v1/issuer/documents/<document-id>/visibility/public

# y volverlo privado
curl -s -X PUT -H "Authorization: Bearer $API_KEY" \
  $API/api/v1/issuer/documents/<document-id>/visibility/private
```

Response — 200 OK

El mismo objeto que devuelve la emisión, con **visibility** actualizada. Todos los campos y su significado están en [la tabla de §4](#).

Campo	Qué cambia
visibility	El valor nuevo, el que se pidió en el path
Todo el resto	Idéntico. La firma, el digest , el credential y el tx_hash no se tocan : el documento no se reemite

Qué controla este endpoint, y qué no

Lo importante de esta llamada no es lo que hace, sino lo que **no** hace:

Cambia	No cambia
Si la zona pública (GET /api/v1/documents/{id}) responde el documento o pide sesión	Las copias del credential ya entregadas, que siguen legibles y verificables para siempre
Nada más. El credential , la firma y el anclaje quedan intactos	El documento en sí: visibility no está cubierta por la firma

Volver a **private** cierra el enlace, no las copias. Ver [§9](#).

Errores de este endpoint

Respuesta	Causa
401 invalid_api_key	Key inválida, o el esquema no era literalmente Bearer
404 document_not_found	Id inexistente, malformado, o de otro workspace
422 invalid_visibility	El último segmento del path no es public ni private

6. GET /api/v1/issuer/documents — Listar los documentos del workspace

Qué hace

Devuelve **todos** los documentos del workspace, del más reciente al más antiguo.

Sin paginación: no hay **?page**, **?limit** ni cursor, y los query params se ignoran. La respuesta crece con el workspace. Tampoco hay filtros ni búsqueda, lo que en la práctica convierte a este endpoint en el único punto de consulta del lado emisor.

Autenticación

Authorization: Bearer <API_KEY>

Request

Sin parámetros, sin body.

```
curl -s -H "Authorization: Bearer $API_KEY" \
  $API/api/v1/issuer/documents
```

Response — 200 OK

```
{
  "documents": [
    { "id": "<uuid>", "status": "anchored", "...": "..." }
  ]
}
```

Campo	Tipo	Qué es
documents	array<object>	Una entrada por documento, ordenadas de la más nueva a la más vieja
documents[]	object	Cada entrada tiene exactamente la misma forma que la respuesta de emisión , incluidos el credential completo y los claims decodificados. Los campos y su significado, en la tabla de §4

Que cada entrada traiga el **credential** completo es lo que hace útil este endpoint: alcanza una llamada para reconstruir todos los documentos emitidos, sin pedir cada uno por separado.

Errores de este endpoint

Respuesta	Causa
401 invalid_api_key	Key inválida, o el esquema no era literalmente Bearer
409 workspace_not_provisioned	El workspace todavía no tiene DID ni claves

7. GET /api/v1/documents/{id} — Leer un documento (público)

Qué hace

Devuelve el documento a quien recibió el enlace. Es el endpoint que consume el destinatario, no el emisor.

Dos diferencias con la zona emisor, las dos visibles en el path: **no hay /issuer** y **no lleva API key**. Si el documento es público, tener el id es toda la autorización. Uno privado necesita sesión del dashboard, que una API key no puede suplir.

Responde con **Access-Control-Allow-Origin: ***, así que un navegador en cualquier dominio puede llamarlo directo, siempre que no envíe credenciales.

Autenticación

Request

Parámetro	Posición	Tipo	Qué es
<code>id</code>	path	<code>string</code> (uuid)	El <code>id</code> del documento

```
curl -s $API/api/v1/documents/<document-id>
```

Response — 200 0K

```
{
  "id": "<uuid>",
  "credential": "eyJhbGciOiJIJFUi1NiIsInR5cCI6IkpzK3Nk...eyJpc3M...~",
  "issuer": { "did": "did:sovra:0x...", "name": "...", "address": "0x..." },
  "schema": { "name": "...", "claims": [ "..." ] },
  "layout": { "...": "..." },
  "anchor": {
    "status": "anchored", "tx_hash": "0x...", "anchored_at": "..."
  }
}
```

Qué responde, campo por campo — lectura pública

Campo	Tipo	Qué es	Firmado
<code>id</code>	<code>string</code> (uuid)	El identificador del documento, el mismo que va en la URL	Sí (como <code>jti</code>)
<code>credential</code>	<code>string</code>	★ El documento en sí. Lo único verificable de toda la respuesta	Es la firma
<code>issuer.did</code>	<code>string</code>	El DID del emisor (<code>did:sovra:0x...</code>); con él se resuelve su clave pública on-chain	Sí (como <code>iss</code>)
<code>issuer.address</code>	<code>string</code> (0x...)	La dirección del emisor en la cadena	Sí, se deriva de <code>iss</code>
<code>issuer.name</code>	<code>string</code>	Nombre visible del emisor. Solo informativo	No
<code>schema.name</code>	<code>string</code>	Nombre humano del schema. Versión reducida: acá no vienen el UUID ni el <code>credential_type</code>	No

Campo	Tipo	Qué es	Firmado
<code>schema.claims[]</code>	<code>array<object></code>	Definición de cada campo: <code>key</code> , <code>label</code> , <code>type</code> , <code>required</code> , <code>disclosable</code> — igual que en la tabla de §4	No
<code>layout</code>	<code>object</code>	Instrucciones de cómo dibujar la hoja — ver abajo	No
<code>anchor.status</code>	<code>string</code>	<code>anchored</code> = la firma quedó registrada on-chain	No
<code>anchor.tx_hash</code>	<code>string</code> (0x...)	Hash de la transacción del anclaje	No
<code>anchor.anchored_at</code>	<code>string</code> (ISO 8601)	Cuándo quedó anclada	No

layout — existe para que un tercero pueda mostrar la hoja con el diseño del emisor. No está firmado:

Campo	Tipo	Qué es
<code>layout.page</code>	<code>object</code>	Dimensiones de la hoja: 794 × 1123 px, la proporción A4 a 96 dpi
<code>layout.elements[]</code>	<code>array<object></code>	Cada elemento visual de la página
<code>elements[].type</code>	<code>string</code>	"text" = texto fijo (títulos, etiquetas). "field" = campo dinámico
<code>elements[].bind</code>	<code>string</code>	Solo en <code>type: "field"</code> : qué claim se dibuja en ese lugar
<code>elements[].x / .y</code>	<code>number</code>	Posición en la página, en píxeles
<code>elements[].size / .color / .bold</code>	<code>number / string / boolean</code>	Estilo del texto
<code>elements[].id</code>	<code>string</code>	Identificador interno del elemento

Un render fiel recorre `layout.elements[]`, dibuja los `text` tal cual y resuelve cada `field` contra el claim que indica su `bind`.

Diferencias contra la respuesta de emisión

Solo en la emisión (§4)	Solo en la lectura pública (§7)
<code>claims</code> decodificados	<code>layout</code>
<code>visibility</code>	
<code>signed_at</code> , <code>revoked_at</code>	
<code>digest</code> , <code>tx_hash</code> en la raíz	Los mismos datos, dentro de <code>anchor</code>
<code>schema.id</code> , <code>schema.schema_id</code> , <code>schema.credential_type</code>	

La lógica del recorte: la emisión responde al emisor, que necesita el estado interno del documento; la lectura pública responde a quien lo recibió, que necesita **poder dibujarlo**. De ahí que este endpoint devuelva **layout** y el otro no.

Errores de este endpoint

Respuesta	Causa
401 authentication_required	El documento es privado y no hay sesión. Se devuelve incluso cuando el header traía una API key
404 document_not_found	Id inexistente, malformado, o de otro workspace. No distingue entre los tres casos , a propósito: no filtra información

Transversal

8. El campo **credential** en detalle

Qué es

Un **SD-JWT VC**: **header.payload.firma~**, cada parte en base64url, separadas por puntos. El sobre sellado tiene tres cosas: el contenido (los claims), la firma digital del emisor —hecha con su clave privada, que solo él tiene— y los metadatos de seguridad: quién lo emitió, cuándo, cuándo expira, su id único.

Es **autocontenido**. Por cualquier vía que se entregue, quien lo recibe puede comprobar que es auténtico **sin preguntarle a nadie**, ni siquiera al servicio que lo creó.

No está cifrado — está codificado

Conviene decirlo explícito porque suele sorprender: **cualquiera puede leer el contenido de un credential**. Base64url no es cifrado.

Header — cómo está firmado:

```
{ "alg": "ES256", "typ": "vc+sd-jwt" }
```

Payload — el documento en sí, legible tal cual:

```
{  
  "content": "Contenido de Prueba",  
  "date": "20-08-2026",  
  "idDocument": 1234567,  
  "isConfidential": false,  
  "iss": "https://test-api-sovra.flagonsa.com/did:sovra:0xb516d2...",  
  "jti": "2a032843-5c82-4244-b555-771e3dd41665",  
  "vct": "Document",  
  "iat": 1787271088,  
}
```

```
"exp": 1818807088
}
```

Los claims del schema van al mismo nivel que los metadatos estándar:

Claim	Qué es
(los del schema)	El contenido del documento, con las key que declaró el schema
iss	El emisor, con su DID embebido
jti	El id del documento — el mismo id de las respuestas de la API
vct	El tipo de credencial — el schema.credential_type
iat	Cuándo fue emitido (epoch)
exp	Cuándo expira (epoch)

En este ejemplo **iat** = 2026-08-21T00:11:28Z y **exp** = 2027-08-21T00:11:28Z: **un año de validez**.

Firma — 64 bytes, el **rlls** de ES256:

```
c66635fb6befeb82a4b93ea8954629cf320026dc340ade0c5ae5e4f7454a17cc
0ac9c733fd365803bebdafcc346ad010411dec38b746ed48086807e620c0707
```

Ese es exactamente el valor que se compara contra lo anclado en la cadena.

La ~ final es parte del formato SD-JWT: después de ella irían los *disclosures* (campos divulgados por separado). En un documento cuyos claims van directo en el payload está vacía.

Las dos capas de confianza

Toda respuesta de esta API se divide en dos, y la distinción es lo más importante de este documento:

Firmado y verificable	Solo afirmación del servicio
Los claims que declaró el schema	issuer.name
vct (el tipo)	visibility
iss (el emisor)	layout y todo lo relativo a cómo se ve la página
jti (el id del documento)	status , digest , tx_hash y los tiempos que informa el servicio
iat / exp	Todos los metadatos alrededor de credential

La columna izquierda vive **dentro** del credential y está cubierta por la firma. La derecha la dice el servidor. Una integración que le dé valor legal al documento debería renderizar y decidir a partir de los claims del credential verificado, no de nada más en la respuesta.

Cómo decodificarlo

Con cualquier decodificador de base64url. En Python:

```
import base64, json

jwt = credential.rstrip('~')
header_b64, payload_b64, sig_b64 = jwt.split('.')

def b64url_decode(s):
    return base64.urlsafe_b64decode(s + '=' * (-len(s) % 4))

header = json.loads(b64url_decode(header_b64))
payload = json.loads(b64url_decode(payload_b64))
```

O pegando el string en jwt.io, quitando la ~ final.

Tres consecuencias prácticas

1. **Leer ≠ verificar.** Decodificar muestra el contenido y no dice absolutamente nada sobre autenticidad. Se puede fabricar un payload idéntico con otros datos; lo que no se puede fabricar es una firma válida. Decodificar sirve para mostrar, nunca para decidir.
2. **Acá no hay secretos.** El contenido es legible por cualquiera que tenga el string. Nunca poner datos sensibles asumiendo que van "ocultos": la confidencialidad la da **quién tiene la copia**, no el formato.
3. **Un documento adulterado se ve idéntico a uno legítimo** si no se comprueba la firma. Alterar un claim y dejar la firma intacta pasa cualquier control que no sea criptográfico. Lo único que los distingue es la verificación contra la cadena.

9. Visibilidad: qué restringe y qué no

Es el malentendido más caro posible en esta API, así que va aparte.

visibility restringe los endpoints, no el artefacto.

- **public** → tener el id/enlace alcanza para leer el documento por la zona pública.
- **private** → el enlace por sí solo no alcanza; hace falta sesión del dashboard.

Lo que **no** hace:

Suposición	Realidad
"Privado = los bytes son secretos"	No. Un credential ya entregado sigue siendo legible y verificable por quien lo tenga, para siempre
"Volver a privado revoca el documento"	No. Cierra el enlace, no las copias. Para invalidar el documento existe la revocación (revoked_at)
"La visibilidad es parte del documento"	No. No está cubierta por la firma. Es un atributo del servicio

En resumen: privado significa *"el enlace por sí solo no basta"*, nunca *"los bytes son secretos"*.

Corolario para quien construya un buscador o índice encima de esta API: servir un documento privado por búsqueda —por id, por digest, por cualquier claim— anula la decisión del emisor por la puerta de atrás. Un índice consultable por terceros debería incluir **solo** documentos con `visibility: "public"`.

10. Verificación: por qué no hay un endpoint

No existe un endpoint que responda "válido" o "inválido", y es a propósito.

Un veredicto habría que creerlo; una firma se puede comprobar. Si la API respondiera `{ "valid": true }`, el destinatario estaría confiando en el mismo servicio que emitió el documento — exactamente la dependencia que el anclaje on-chain viene a eliminar. La verificación corre entera del lado de quien verifica, contra la cadena.

Lo que la API aporta para eso está en los campos ya descritos:

Campo	Para qué sirve al verificar
<code>credential</code>	Contiene el payload, la firma y el DID del emisor. Es la entrada de todo el proceso
<code>issuer.did</code>	Con él se resuelve la clave pública del emisor en el registry on-chain
<code>digest</code>	Se recalcula sobre los bytes del credential y se compara
<code>tx_hash</code>	Ubica el registro de la firma anclada en la cadena

Qué cubre la firma y qué no, en [las dos capas de confianza](#).

11. Tabla completa de errores

Respuesta	Endpoints	Causa	Qué revisar
<code>401 invalid_api_key</code>	§4, §5, §6	Key desconocida, malformada o revocada — o el esquema no era literalmente Bearer	Que el header diga Bearer con B mayúscula
<code>401 authentication_required</code>	§7	Documento privado, sin sesión. Se devuelve incluso cuando el header traía una API key	Publicar el documento (§5), o abrirlo desde el dashboard
<code>404 document_not_found</code>	§5, §7	Id inexistente, malformado, o de otro workspace. No distingue entre los tres casos , a propósito	El id y el workspace al que pertenece la key
<code>409 workspace_not_provisioned</code>	§4, §6	El workspace todavía no tiene DID ni claves	Provisionar el workspace desde el dashboard

Respuesta	Endpoints	Causa	Qué revisar
422 wrong_schema_kind	§4	El schema es kind: "credential" (con holder), no "document"	Usar un schema de documentos
422 missing_claims	§4	Falta un claim requerido. La respuesta lista cuáles	Los required de schema.claims[]
422 invalid_visibility	§4, §5	El valor no es public ni private	El último segmento del path, o el campo del body
503 chain_not_configured	§4	No hay registro de firmas configurado: se niega a firmar	Es intencional — prefiere fallar antes que emitir algo no anclable

12. Colección Postman

Archivo: **Sovra – Documentos Firmados.postman_collection.json** (formato v2.1.0).

Variables de colección

Variable	Valor	Notas
base_url	URL base de la API, sin slash final	
api_key	La key del workspace	Solo la usan los requests de /issuer . Vaciarla antes de exportar o compartir la colección
schema_id	Id del schema de tipo document	
document_id	Id del documento	Se autocompleta al emitir

Requests

Carpeta "Issuer (requiere API key)" — todos con **Authorization: Bearer {{api_key}}**:

Request	Llamada	Sección
Emitir documento	POST {{base_url}}/api/v1/issuer/documents	§4
Cambiar visibilidad → public	PUT .../issuer/documents/{{document_id}}/visibility/public	§5
Cambiar visibilidad → private	PUT .../issuer/documents/{{document_id}}/visibility/private	§5

Request	Llamada	Sección
Listar documentos del workspace	GET <code>{{base_url}}/api/v1/issuer/documents</code>	§6

Carpeta "Público (sin API key)" — sin headers de autenticación:

Request	Llamada	Sección
Obtener documento (JSON)	GET <code>{{base_url}}/api/v1/documents/{{document_id}}</code>	§7

Auto-encadenado

El request de emisión trae un test script: si la respuesta es **201**, guarda `body.id` en la variable `document_id`. Por eso el orden natural de uso es **emitir** → **cambiar visibilidad** → **leer**, sin copiar ids a mano.

Tests incluidos

Request	Verifica
Emitir documento	201 Created , y que <code>credential</code> sea un string
Visibilidad → public / private	200 OK , y que <code>visibility</code> quedó en el valor pedido
Listar documentos	200 OK , y que <code>documents</code> sea un array
Obtener documento (JSON)	Que el código sea 200 o 401 —según si el documento está publicado— y, si es 200 , que traiga <code>credential</code>

Ese último test acepta los dos códigos a propósito: un **401** acá no es una falla de la integración, es la respuesta correcta para un documento privado.

Documentación de la API de documentos firmados de Sovra. Compilada el 21-08-2026, a partir de la guía de integración oficial, la colección Postman y las pruebas contra el ambiente `test-api-sovra`.